

NutriHogar — Sprint UI 1: Consumer App Shell

Quiero implementar el primer sprint del nuevo rumbo de producto de NutriHogar.

Este sprint debe transformar la **capa visual y de navegación principal** del frontend para que NutriHogar deje de sentirse como un panel administrativo o ERP y empiece a sentirse como una aplicación consumer moderna de nutrición.

MUY IMPORTANTE

Este sprint **NO debe refactorizar todavía los flujos funcionales de negocio**.

No implementar en este sprint:

- nuevo Meal Composer;
- IA para interpretar comidas;
- fotografía de comidas;
- cambios al flujo de registro;
- refactor completo de Home;
- cambios a recetas;
- cambios funcionales a inventario;
- cambios funcionales a planificación;
- cambios funcionales a compras;
- nuevas entidades backend;
- modificaciones al backend;
- modificaciones al cliente OpenAPI salvo que sean estrictamente necesarias por razones técnicas existentes.

El objetivo es construir la **nueva carcasa consumer de NutriHogar** sobre las funcionalidades que ya existen.

1. Contexto del producto

NutriHogar es una aplicación de nutrición diseñada para personas, parejas y familias.

El núcleo del producto debe sentirse como un contador de calorías moderno, rápido y agradable.

Sobre ese núcleo existen herramientas adicionales de hogar:

- planificación semanal;
- recetas;
- preparaciones familiares;
- inventario;

- compras;
- lista de compras;
- progreso corporal;
- seguimiento nutricional.

La arquitectura del frontend ya está razonablemente separada en:

- domain
- application
- infrastructure
- presentation

También existen:

- cliente OpenAPI generado;
- TanStack Query;
- React Hook Form;
- Zod;
- dependencias centralizadas;
- tokens de diseño.

No queremos destruir esa arquitectura.

Queremos cambiar **cómo se presenta el producto al usuario**.

2. Nuevo principio de producto

NutriHogar debe sentirse primero como:

Una aplicación personal de nutrición que además entiende que las personas viven, cocinan y comen en familia.

El uso cotidiano debe sentirse cercano a aplicaciones consumer como:

- Fitia;
- Cal AI;
- Welmi;

pero **NO copiar interfaces, branding, layouts ni colores exactamente**.

Tomar de estas referencias principalmente:

- claridad;
- jerarquía fuerte;
- información inmediata;
- rapidez;

- sensación de producto consumer;
- poco ruido;
- acciones principales muy evidentes;
- uso expresivo del color;
- interfaces agradables en móvil.

NutriHogar debe desarrollar su propia identidad.

3. Problema actual

La auditoría detectó que aunque la arquitectura interna es razonable, la experiencia visual expone directamente demasiados bounded contexts.

Actualmente existen conceptos como:

- alimentos;
- comidas;
- recetas;
- preparaciones;
- sobrantes;
- inventario;
- compras;
- lista de compras;
- planificación;
- adherencia;
- requerimientos;
- comparación de inventario;

que visualmente compiten demasiado entre sí.

Eso produce una experiencia cercana a:

- ERP;
- dashboard SaaS;
- aplicación administrativa.

El nuevo shell debe ocultar esa complejidad.

4. Objetivo del sprint

Crear una nueva estructura global donde el usuario perciba únicamente cinco grandes áreas:

1. **Hoy**
2. **Plan**

- 3. **Registrar**
- 4. **Progreso**
- 5. **Hogar**

Estas serán las únicas áreas primarias de navegación.

Las rutas profundas y funcionalidades existentes pueden mantenerse internamente.

La navegación debe agruparlas bajo estas cinco intenciones.

5. Navegación móvil

La aplicación es mobile-first.

En móvil utilizar una barra inferior persistente con aproximadamente:



El botón central  representa **Registrar comida**.

Debe ser la acción visualmente más importante de la navegación.

En este sprint:

- puede continuar llevando al flujo de registro existente;
- NO hay que construir todavía el nuevo Meal Composer.

El diseño debe dejar preparada la navegación para que en el siguiente sprint el botón central pueda abrir el nuevo composer.

Requisitos

La bottom navigation debe:

- ser cómoda desde 320 px;
- tener estados activos inequívocos;
- tener labels visibles;
- no depender únicamente de iconos;
- respetar safe areas;
- ser accesible con teclado cuando aplique;
- utilizar `aria-current` correctamente;
- no tapar contenido;
- integrarse con scroll;

- soportar rutas profundas;
- conservar contexto cuando se navega entre áreas.

6. Navegación desktop

En escritorio mantener una navegación lateral, pero simplificada.

Debe mostrar solamente:

- Hoy
- Plan
- Registrar
- Progreso
- Hogar

No mostrar permanentemente:

- Preparaciones
- Sobrantes
- Compras
- Inventario
- Lista de compras
- Requerimientos
- Comparación de inventario
- Adherencia
- etc.

Estos elementos deben aparecer como navegación secundaria contextual cuando corresponda.

Ejemplo conceptual

NUTRIHOGAR

Hoy

Plan

Registrar

Progreso

Hogar

El sidebar debe ser:

- limpio;
- compacto;
- visualmente ligero;
- sin exceso de secciones;
- sin aspecto de admin dashboard.

7. Agrupación de rutas

Mantener deep links existentes siempre que sea razonable.

Agrupar conceptualmente:

Hoy

- `/app`
- vistas relacionadas con el día actual.

Plan

- planificación semanal;
- comidas planificadas;
- requerimientos;
- comparación con inventario;
- adherencia relacionada con planificación;
- rutas profundas de planificación.

Progreso

- resumen nutricional;
- reportes;
- peso;
- medidas;
- tendencias;
- adherencia personal;
- síntomas cuando se incorporen.

Hogar

- integrantes;
- perfil familiar;
- inventario;
- lista de compras;
- compras;
- preparaciones;
- sobrantes;
- configuración del hogar.

Registrar

- registro actual de comidas.

No es necesario cambiar URLs solo para que coincidan con esta organización.

La arquitectura visual no debe depender de que las URLs sean perfectas.

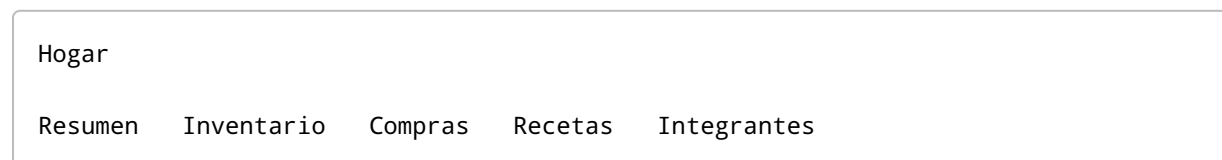
8. Section navigation

Cuando el usuario entra a una de las áreas complejas:

- Plan;
- Progreso;
- Hogar;

puede aparecer navegación secundaria contextual.

Ejemplo Hogar:



No necesariamente debe implementarse exactamente como tabs.

Elegir el patrón más apropiado según viewport:

- tabs;
- segmented navigation;
- horizontal scroll;
- contextual sidebar;
- dropdown;

- section header.

No introducir un segundo sidebar pesado.

9. Dirección visual

Queremos abandonar la estética:

- administrativa;
- demasiado neutra;
- exceso de cards;
- demasiados bordes;
- exceso de pequeñas acciones;
- layouts tipo dashboard;
- componentes que parecen de backoffice.

La nueva estética debe sentirse:

- fresca;
 - energética;
 - amigable;
 - moderna;
 - humana;
 - ligeramente juguetona;
 - cuidada;
 - premium sin parecer lujosa;
 - cercana a alimentación y bienestar sin parecer clínica.
-

10. Libertad creativa

NO quiero que te limites a mantener la apariencia actual.

Puedes proponer una evolución significativa del lenguaje visual.

Pero:

- conserva accesibilidad;
- conserva consistencia;
- evita modas visuales que reduzcan usabilidad;
- evita glassmorphism excesivo;
- evita gradients decorativos por todas partes;
- evita sombras grandes;
- evita elementos simplemente “bonitos” sin propósito.

Priorizar:

producto antes que decoración.

11. Color

NutriHogar necesita más personalidad visual.

Actualmente se utiliza verde, pero no debe mantenerse automáticamente el verde actual si resulta demasiado apagado.

La nueva identidad puede explorar un **verde/lima vivo y reconocible** como color de marca.

No copiar exactamente Fitia.

No convertir toda la interfaz en verde.

El color principal debe utilizarse principalmente para:

- CTA principal;
- navegación activa;
- indicadores de progreso;
- selecciones;
- pequeños highlights;
- feedback positivo;
- elementos importantes.

La mayoría de superficies deben mantenerse neutrales.

Crear una paleta coherente mediante tokens.

Debe contemplar como mínimo:

```
brand
brand-hover
brand-active
brand-soft

background
surface
surface-secondary

text-primary
text-secondary
```

text-muted

border
border-strong

success
warning
danger
info

Evitar valores hexadecimales repetidos directamente en componentes.

Centralizarlos en design tokens.

12. Tipografía

La auditoría indica que actualmente se utiliza Inter.

Evaluar sustituirla por una tipografía con algo más de personalidad consumer.

Preferencias posibles:

- Manrope;
- Plus Jakarta Sans;
- Geist;

pero puedes recomendar otra si existe una razón clara.

Debe seguir siendo:

- muy legible;
- adecuada para números;
- adecuada para interfaces densas;
- eficiente en móvil.

Definir jerarquía global consistente para:

- display / métricas grandes;
- page title;
- section title;
- item title;
- body;
- label;
- metadata;
- helper text.

No crear tamaños arbitrarios página por página.

13. Números

NutriHogar muestra constantemente:

- calorías;
- gramos;
- macros;
- peso;
- cantidades;
- porcentajes.

Los números son parte importante de la identidad.

Optimizar:

- `font-variant-numeric`;
- alineación;
- contraste;
- tamaño;
- legibilidad.

Los números importantes deben poder destacar sin depender de cards gigantes.

14. Spacing

Definir una escala consistente.

No utilizar márgenes arbitrarios constantemente.

Buscar una densidad media:

- más aire que un ERP;
- menos aire que una landing page.

La aplicación debe permitir escanear varias comidas o productos sin hacer scroll excesivo.

15. Cards

Regla importante:

No todo debe ser una card.

Preferir:

- jerarquía tipográfica;
- espacio;
- divisores;
- agrupación;
- backgrounds suaves;
- filas;
- listas.

Usar cards solo cuando exista una entidad o agrupación claramente delimitada.

Eliminar la sensación actual de:

card
card
card
card
card

16. Elevación

Las sombras deben comunicar profundidad real.

Usarlas principalmente en:

- bottom sheets;
- drawers;
- popovers;
- menus;
- elementos flotantes.

Las superficies normales deben depender principalmente de:

- fondos;
 - borders suaves;
 - separación.
-

17. Bordes

Evitar contornos demasiado prominentes.

Inputs y superficies pueden utilizar bordes suaves.

La interfaz no debe sentirse construida con rectángulos encerrados.

18. Radius

Definir tokens para radius.

Buscar una sensación consumer moderna.

Ejemplo orientativo:

- control: 10–12 px;
- surface: 14–16 px;
- overlay: 20–24 px;
- pill: completo.

No es obligatorio usar estos valores exactos.

19. Botones

Crear o revisar jerarquía:

Primary

Acción principal del contexto.

Secondary

Acción complementaria.

Tertiary / ghost

Acción poco prominente.

Destructive

Acción destructiva.

No debe haber cuatro botones visualmente equivalentes compitiendo en una pantalla.

El CTA de marca debe tener personalidad.

20. Iconografía

Auditar la librería actual.

Buscamos:

- iconos modernos;
- coherentes;
- simples;
- algo amigables;
- no excesivamente técnicos.

No utilizar iconos solos cuando el significado no sea inequívoco.

No cambiar toda la librería únicamente por estética si eso genera un refactor innecesario.

Si propones cambiarla, justificarlo.

21. App header

Revisar el Topbar actual.

En móvil debe ocupar muy poco espacio.

Evitar header tipo aplicación empresarial.

Debe contener solamente lo imprescindible:

- marca/contexto;
- perfil cuando sea útil;
- acciones realmente globales.

No duplicar información que ya esté visible en la pantalla.

22. Perfil e integrante activo

La auditoría detectó que la selección de perfil está dispersa entre páginas.

En este sprint:

Crear un **contexto visual y de estado coherente para el integrante activo**.

No reescribir innecesariamente el dominio.

Queremos que:

- Hoy;
- Registrar;
- Progreso;

puedan conocer claramente qué integrante está activo.

Cuando exista un único integrante activo, no mostrar controles innecesarios.

Cuando haya varios:

mostrar un selector sencillo.

Ejemplo conceptual:

Alejandro ▾

Al abrirlo:

- Alejandro
- Camila

En móvil debería resolverse preferentemente mediante bottom sheet.

23. Household context

El hogar activo también debe permanecer coherente.

Evitar repetir selectores de hogar en todas partes.

Solo mostrar selector cuando:

- exista más de un hogar;
 - sea realmente necesario.
-

24. Sistema común de overlays

La auditoría detectó inconsistencia en modales.

Crear primitivas reutilizables para:

Dialog

Para:

- decisiones cortas;
- confirmaciones;
- contenido pequeño.

BottomSheet

Principal primitive móvil para:

- selectores;
- acciones contextuales;
- filtros;
- selección de perfil;
- futuras acciones de Meal Composer.

Drawer

Equivalente desktop cuando tenga sentido.

Deben compartir:

- backdrop;
- cierre;
- escape;
- focus trap;
- devolución de foco;
- títulos accesibles;
- manejo de scroll;
- animaciones;
- reduced-motion.

NO migrar todos los modales de la app en este sprint salvo aquellos necesarios para validar estos primitivos.

Construir la base reutilizable.

25. Motion

Agregar motion muy discreto.

Rango orientativo:

150–250 ms.

Utilizar para:

- apertura de navigation;
- bottom sheets;
- selección;
- estados;
- hover;
- expansión.

Respetar:

`prefers-reduced-motion`.

No crear animaciones decorativas innecesarias.

26. Estados de aplicación

La auditoría detectó mensajes inconsistentes como:

- Cargando...
- No se pudo cargar...
- errores técnicos directos.

Crear una estrategia compartida para:

LoadingState

ErrorState

EmptyState

OfflineState / ConnectionNotice

Los errores de infraestructura no deben mostrarse literalmente cuando no ayuden al usuario.

Ejemplo:

Evitar:

```
Request failed with status 500
```

Preferir:

```
No pudimos cargar esta información.
```

```
Reintentar
```

Los logs técnicos pueden continuar disponibles internamente.

27. Empty states

Evitar estados vacíos genéricos.

El componente debe soportar:

- título;
- descripción;
- CTA opcional;
- contenido visual opcional.

Ejemplo futuro:

```
Todavía no registraste comidas hoy.
```

```
Empieza con lo que desayunaste.
```

Registrar comida

No crear ilustraciones complejas en este sprint salvo que exista una razón.

28. PageHeader

La auditoría detectó que `PageHeader .tsx` acepta `eyebrow` pero actualmente no lo renderiza.

Corregir esta inconsistencia.

Pero también revisar si todas las páginas realmente necesitan un PageHeader grande.

En mobile consumer:

- algunos headers deben ser muy compactos;
- evitar repetir títulos evidentes.

Crear variantes si es necesario.

29. Page shell

Crear una estructura de página consistente.

Debe contemplar:

- ancho máximo adecuado;
- padding responsive;
- header;
- contenido;
- optional action;
- optional subnavigation.

No hacer que cada módulo reconstruya esto independientemente.

30. Responsive

Validar mínimo en:

Mobile

320 px

390 px

Tablet

768 px

Desktop

1440 px

No limitarse a verificar que “no se rompa”.

Revisar:

- densidad;
- navegación;
- tamaño de targets;
- line length;
- overlays;
- sticky elements;
- scroll;
- orientación.

31. Touch targets

Todos los controles interactivos importantes deben tener un target cómodo en móvil.

Objetivo:

aproximadamente 44 px o superior cuando sea razonable.

32. Accesibilidad

Mantener o mejorar:

- contraste;
- navegación por teclado;
- focus visible;

- semantic HTML;
- labels;
- aria;
- headings;
- landmarks;
- focus management;
- reduced motion.

No sacrificar accesibilidad por estética.

33. CSS

La auditoría detectó CSS huérfano en:

```
modules/home/presentation/home.css
```

incluyendo estilos como:

- `.home-page__planned-meals`
- `.home-page__summary`
- `.home-page__organization`
- `.home-pulse-grid`
- `.organization-links`
- `.household-section`
- `.home-actions`

En este sprint:

- limpiar estilos claramente huérfanos;
- evitar mantener CSS muerto;
- reducir duplicación;
- mover valores globales a tokens cuando corresponda;
- evitar un refactor masivo de todos los CSS de módulos que no sea necesario.

No convertir todo el proyecto a otra tecnología de styling.

34. Design tokens

Revisar:

```
packages/design-tokens/src/tokens.css
```

Debe convertirse en la fuente principal del lenguaje visual global.

Organizar tokens aproximadamente en:

```
colors
typography
spacing
radius
shadows
motion
layout
```

No sobreingenierizar.

Crear únicamente los tokens que realmente se utilicen.

35. Componentes base esperados

Auditar los existentes antes de crear nuevos.

Idealmente el shell debería terminar con primitivos similares a:

```
AppShell
AppSidebar
MobileBottomNavigation
SectionNavigation
PageContainer
PageHeader

Button
IconButton

Dialog
BottomSheet
Drawer

LoadingState
ErrorState
EmptyState
ConnectionNotice

MemberSelector
```

No es obligatorio utilizar exactamente estos nombres.

Seguir las convenciones actuales del proyecto.

36. No duplicar componentes

Antes de crear cualquier primitive:

buscar si ya existe una implementación equivalente.

Si existe:

refactorizarla.

No crear:

```
Button  
NewButton  
ConsumerButton  
ActionButton
```

para resolver el mismo problema.

37. Rutas profundas

Muy importante:

El refactor visual NO debe romper deep links existentes.

Ejemplos:

- detalle de receta;
- preparación;
- inventario;
- compras;
- edición de comida.

Estas páginas pueden seguir existiendo aunque desaparezcan de la navegación principal.

38. Backwards compatibility

Después del sprint:

Las funcionalidades existentes deben continuar siendo accesibles.

Este sprint cambia:

cómo llegamos a ellas

más que:

cómo funcionan internamente.

39. No refactorizar Home todavía

Muy importante.

No reconstruyas todavía `HomePage.tsx` como el nuevo dashboard consumer.

Solo realizar los cambios mínimos necesarios para que funcione dentro del nuevo App Shell.

El rediseño real de Hoy será otro sprint.

40. No crear Meal Composer todavía

El botón `Registrar` debe integrarse visualmente con la nueva navegación.

Pero continuar utilizando temporalmente el registro actual.

No implementar:

- MealDraft;
- nueva búsqueda;
- recipes as source;
- AI;
- photo;
- recent meals;

en este sprint.

Eso será Sprint UI 2.

41. Archivos a inspeccionar primero

Revisar al menos:

```
apps/web/src/app/router/appRoutes.tsx
apps/web/src/app/composition/dependencies.ts

PrivateLayout.tsx
Sidebar.tsx
Topbar
Mobile navigation
SectionNavigation.tsx
mainNavigation.tsx

PageHeader.tsx

packages/design-tokens/src/tokens.css

apps/web/src/globals.css

modules/home/presentation/home.css
```

Y cualquier componente relacionado que descubras.

No asumir rutas exactas si alguno ha cambiado.

Buscar primero.

42. Flujo de trabajo obligatorio

Antes de modificar:

1. Inspecciona la implementación actual.
2. Identifica primitives existentes reutilizables.
3. Identifica tests actuales.
4. Identifica rutas que dependen del shell.
5. Identifica CSS global y modular afectado.
6. Define internamente una estrategia de cambio mínima.
7. Después implementa.

No hagas un rewrite completo del frontend.

43. Criterios de aceptación funcionales

El sprint está completo cuando:

Navegación

- existen únicamente cinco áreas principales;
- Hoy;
- Plan;
- Registrar;
- Progreso;
- Hogar.

Mobile

- existe nueva bottom navigation;
- Registrar es la acción central;
- funciona desde 320 px;
- no tapa contenido.

Desktop

- sidebar compacto;
- solo cinco áreas;
- rutas secundarias no saturan navegación.

Deep links

- rutas existentes siguen funcionando.

Contexto

- hogar activo continúa funcionando;
- integrante activo tiene un comportamiento coherente;
- no aparecen selectores redundantes.

Visual

- el producto deja de sentirse como dashboard administrativo;
- existe una identidad visual consumer consistente;
- la paleta tiene mayor personalidad;
- la tipografía es coherente;
- disminuye el uso innecesario de cards;
- spacing consistente.

Overlays

- existe primitive de BottomSheet;
- Dialog consistente;
- focus management correcto.

Estados

- Loading;
- Empty;
- Error;

tienen componentes coherentes.

Tokens

- colores globales centralizados;
- spacing/radius/motion principales centralizados.

Accessibility

- navegación usable con teclado;
- focus visible;
- labels;
- contraste adecuado;
- reduced motion.

44. Testing requerido

Actualizar o crear pruebas para:

Routing

- cada área principal;
- deep links;
- estado activo;
- navegación móvil;
- navegación desktop.

App Shell

- mobile;
- desktop;
- contenido;
- sidebar;
- bottom navigation.

Overlays

- abrir;
- cerrar;
- escape;
- backdrop;
- focus;
- restauración de focus.

Member selector

- un miembro;
- múltiples miembros.

State components

- loading;
- error;
- empty.

No modificar tests de negocio innecesariamente.

45. Validación manual requerida

Revisar manualmente en:

320 px
390 px
768 px
1440 px

Validar al menos:

- login → aplicación;
- Hoy;
- Plan;
- Registrar;
- Progreso;
- Hogar;
- detalle de receta;
- inventario;
- compras;
- ruta profunda de planificación.

46. Calidad técnica

Al terminar ejecutar:

```
lint
tests
build
```

Usar los comandos reales definidos por el repositorio.

Resolver fallos introducidos por el refactor.

No ignorar warnings importantes.

47. Entrega final

Al terminar, entregar un informe con:

Resumen

Qué cambió visual y estructuralmente.

Arquitectura

Qué primitivos nuevas fueron creadas.

Navegación

Cómo quedan agrupadas las rutas.

Design system

Qué tokens fueron modificados.

Archivos

Listado de archivos creados/modificados/eliminados.

Compatibilidad

Qué funcionalidades existentes fueron verificadas.

Testing

Comandos ejecutados y resultados.

Revisión manual

Pantallas/viewports comprobados.

Deuda detectada

Cualquier problema encontrado que deba resolverse en Sprint UI 2.

48. Fuera de alcance estricto

No implementar aunque parezca una mejora natural:

- Meal Composer;
- Meal Draft;
- interpretación IA;
- fotografía;
- voz;
- comidas frecuentes;
- nuevo backend;
- SavedMeal;
- nuevas APIs;
- rediseño total de Home;
- refactor de Recipe workflow;
- refactor de PreparedBatch;
- refactor completo del inventario;
- refactor completo de planificación;
- nuevos reportes;
- cambios a cálculos nutricionales.

Si encuentras una necesidad relacionada:

documentarla, no implementarla.

49. Resultado esperado

Al terminar este sprint quiero poder abrir NutriHogar y sentir que estoy entrando a una **app consumer de nutrición moderna**, aunque debajo todavía existan temporalmente algunas pantallas antiguas.

La primera impresión debe cambiar de:

“Sistema con muchos módulos.”

a:

“Esta es mi aplicación de alimentación diaria.”

La complejidad de recetas, preparaciones, inventario, planificación y compras debe quedar organizada detrás de cinco áreas claras.

No queremos esconder funcionalidades.

Queremos esconder **complejidad innecesaria**.

La prioridad absoluta de este sprint es establecer una base visual y de navegación sólida sobre la cual construiremos en el siguiente sprint:

Meal Composer + registro diario rápido.